

Custom 16-Bit ISA Architecture

ASSEMBLY LANGUAGE & ARCHITECTURE REFERENCE MANUAL

1. System Overview

This architecture represents a custom **16-bit von Neumann processor** designed with unified memory, direct register operand manipulation, and a simplified instruction pipeline.

- **Word Size:** 16 bits (2 bytes per word).
- **General Purpose Registers:** 16 internal registers (`R0` through `R15`).
- **Address Space:** Up to 32,768 (`0x8000`) words of RAM (15-bit addressable space).
- **Flags Register (CPU Status):**
 - `F0` — **Zero Flag (Z)**: Set to `true` if the primary operation result equals 0.
 - `F1` — **Negative Flag (N)**: Set to `true` if the primary operation result is negative (< 0).
 - `F2` — **Overflow Flag (V)**: Set to `true` if a signed arithmetic operation (`ADD` / `SUB`) exceeds standard 16-bit boundaries.

2. Instruction Encoding & Format

Instructions are encoded into fixed 16-bit words divided into four distinct 4-bit nibbles:

Bits 15 – 12	Bits 11 – 8	Bits 7 – 4	Bits 3 – 0
Opcode	Rx (Destination / Target)	Ry (Source 1 / Base)	Rz (Source 2)

- **Opcode:** Defines the operation to execute (4 bits, values `0x0` – `0xF`).
- **Rx:** High nibble operand. Typically serves as the destination register or jump target.
- **Ry:** Mid nibble operand. Specifies the first source register or address pointer.
- **Rz:** Low nibble operand. Specifies the second source register or shift amount.

3. Instruction Set Reference

3.1 Arithmetic & Logic Operations

Mnemonic	Syntax	Opcode	Description	Flags Set
<code>NOP</code>	<code>NOP</code>	<code>0x0</code>	No operation / Halt processor execution.	—
<code>OR</code>	<code>OR Rx, Ry, Rz</code>	<code>0x1</code>	Bitwise OR: $Rx = Ry \mid Rz$	Z, N
<code>NOT</code>	<code>NOT Rx, Ry</code>	<code>0x2</code>	Bitwise NOT: $Rx = \sim Ry$	Z, N

Mnemonic	Syntax	Opcode	Description	Flags Set
XOR	XOR Rx, Ry, Rz	0x3	Bitwise XOR: $Rx = Ry \oplus Rz$	Z, N
ADD	ADD Rx, Ry, Rz	0x4	Signed Addition: $Rx = Ry + Rz$	Z, N, V
SUB	SUB Rx, Ry, Rz	0x5	Signed Subtraction: $Rx = Ry - Rz$	Z, N, V
SFL	SFL Rx, Ry, Rz	0x6	Logical Shift Left: $Rx = Ry \ll (Rz \& 0xF)$	Z, N
SFR	SFR Rx, Ry, Rz	0x7	Logical Shift Right: $Rx = Ry \gg (Rz \& 0xF)$	Z, N
AND	AND Rx, Ry, Rz	0xF	Bitwise AND: $Rx = Ry \& Rz$	Z, N

3.2 Memory Access Operations

Mnemonic	Words	Syntax	Opcode	Description
LDR	1	LDR Rx, Ry	0x8	Register Load: $Rx = RAM[Ry]$
LDI	2	LDI Rx, imm16	0x9	Immediate Load: $Rx = RAM[PC++]$
STR	1	STR Rx, Ry	0xA	Register Store: $RAM[Rx] = Ry$
STI	2	STI Rx, addr16	0xB	Immediate Store: $RAM[addr16] = Rx$

3.3 Control Flow Operations

Mnemonic	Syntax	Opcode	Description
JMP	JMP Rx	0xC	Unconditional Jump: $PC = Rx$
JMZ	JMZ Rx	0xD	Jump if Zero: If $F0 == true$ then $PC = Rx$
JMN	JMN Rx	0xE	Jump if Negative: If $F1 == true$ then $PC = Rx$

4. Program Example & Memory Layout

The following example loads two constants into registers, performs a subtraction operation, and halts execution.

```
; Assembly Source Example
LDI R1, 10      ; Load 10 into R1 (2 words)
LDI R2, 5       ; Load 5 into R2 (2 words)
SUB R3, R1, R2  ; R3 = R1 - R2 (1 word)
NOP            ; Halt execution (1 word)
```

RAM Representation

Memory Address	Machine Code (Hex)	Instruction / Data Mapping
0x0000	0x9100	LDI R1 opcode word

Memory Address	Machine Code (Hex)	Instruction / Data Mapping
0x0001	0x000A	Immediate operand: 10
0x0002	0x9200	LDI R2 opcode word
0x0003	0x0005	Immediate operand: 5
0x0004	0x5312	SUB R3, R1, R2 opcode word
0x0005	0x0000	NOP (Halt flag trigger)